Universidad Mayor de San Andrés
Facultad de Ingeniería
Ingeniería Electrónica

Informe de laboratorio N°6

“Herramientas Para Simulación, Implementación Y Supervisión De Procesos”

Asignatura:

Sistemas de Control II - ETN 902
Aplicación de Técnicas de Control - ETN 1034

Docente:

Ing. Jorge Antonio Nava Amador

Auxiliares:

- Aux. Limber Ajnota Cahuaya (ETN1034)
- Aux. Fabian Plata Vargas (ETN902)

Integrantes Grupo Planta de manufactura:

- Univ. Herrera Tola Gabriel Sebastian (ETN1034)
- Univ. Mamani Tola José Aldin (ETN1034)
- Univ. Monzon Hidalgo Josue Andre (ETN 902)

Fecha de presentación: 14/06/26

La Paz - Bolivia

Resumen **3**
Objetivos **3**
Objetivo General 3
Objetivos Específicos 3
Marco Teórico (Investigación Previa) **3**
Normas para Programación de PLC 3
Lenguajes de programación Gráficos 4
Lenguajes de programación Textuales 5
Metodos de Diseno de Control Secuencial 5
Conceptos de Supervisión de Procesos 6
Justificación De Las Tecnologías Utilizadas **7**
Python (Simulación y HMI) 7
OpenPLC (Control Estandarizado) 7
Modbus TCP (Protocolo de Comunicación) 8
Configuración Del Entorno De Simulación **8**
Arquitectura del Sistema Propuesto 8
Configuración del Dispositivo Remoto en Open PLC Editor 8
Tabla de Direccionamiento Modbus (Mapa de Memoria) 9
Bobinas (Coils - Actuadores / Salidas PLC) 9
Entradas (Discrete Inputs - Sensores / Entradas PLC) 10
Desarrollo Del Laboratorio Y Secuencia De Control **12**
Proceso Secuencial de la Planta 12
Implementación de la Lógica en Structured Text (ST) 13
Pruebas en Modo Manual 14
Pruebas de Parada de Emergencia 14
Validacion Y Analisis De Resultados **15**
Tiempos de Respuesta Cinemáticos 15
Estabilidad de la Comunicación Modbus TCP 15
Limitaciones Y Comparación Con La Planta Real **16**
Propuestas De Mejora Y Trabajo Futuro **16**
Conclusiones **17**

Resumen

El presente laboratorio implementa una simulación virtual de la planta de manufactura mediante una arquitectura Python de tres capas: una planta virtual que modela en tiempo real la parte física neumática y el servomotor de transporte, un servidor Modbus TCP que expone el mapa de memoria de entradas y salidas, y una interfaz HMI web que permite la supervisión del proceso a 10 Hz. El controlador secuencial ejecuta una máquina de 20 estados que gobierna el ciclo completo de producción con un tiempo de ciclo estimado de 13.5 segundos por pieza. Se verificó el funcionamiento en modo automático, manual y ante parada de emergencia, con una latencia de comunicación Modbus menor a 5 ms.

Objetivos

Objetivo General

Aplicar herramientas de simulación de sistemas automaticos para el analisis, diseño, implementacion y validacion de estrategias de control y automatizacion industrial, mediante la integración de un simulador físico desarrollado en Python y un controlador virtual (OpenPLC Runtime) interconectados vía Modbus TCP, validando la supervisión en tiempo real mediante una interfaz HMI web.

Objetivos Específicos

- Configurar y validar el entorno biofísico del gemelo digital de la planta de manufactura flexible XK335B.
- Justificar el uso de tecnologías de programación abierta (Python), control estandarizado (OpenPLC) y protocolos industriales (Modbus TCP) en el desarrollo de gemelos digitales.
- Mapear de forma estructurada las variables físicas de entrada (sensores) y de salida (actuadores) de la planta en el banco de registros Modbus, y sincronizarlas con el direccionamiento local de OpenPLC (%IX y %QX).
- Programar y depurar la lógica de control secuencial utilizando el lenguaje de programación Structured Text (ST) bajo la norma IEC 61131-3, resolviendo problemas de retención de memoria y control de temporizadores.
- Validar la operación de la planta en modo manual y automático a través del panel de supervisión HMI, evaluando la respuesta de seguridad frente a la activación de la parada de emergencia.
- Evaluar de forma crítica los tiempos de respuesta y las limitaciones cinéticas del modelo simulado en comparación con el comportamiento de la planta real XK335B.

Marco Teórico (Investigación Previa)

Normas para Programación de PLC

La automatización industrial moderna exige un alto grado de estandarización para garantizar la portabilidad, modularidad y mantenibilidad del software desarrollado para Controladores Lógicos Programables (PLC). En este contexto, la norma internacional **IEC 61131-3** (administrada por la Comisión Electrotécnica Internacional) se establece como el único estándar global definitivo para la programación de estos dispositivos.

Antes de la adopción masiva de esta norma, cada fabricante de PLC (Siemens, Rockwell, Schneider, etc.) utilizaba su propio entorno y variantes de lenguaje, lo que limitaba drásticamente la compatibilidad. La norma IEC 61131-3 resuelve este problema unificando la arquitectura del software de control y definiendo un conjunto común de tipos de datos, variables y, fundamentalmente, **cinco lenguajes de programación estándar**. Estos lenguajes se clasifican rigurosamente en dos grandes categorías: gráficos y textuales.

Lenguajes de programación Gráficos

- **LADDER / LD (Diagrama de Contactos):** Es el lenguaje gráfico más extendido en la industria mundial. Está directamente inspirado en los esquemas eléctricos de lógica cableada y relés analógicos. Estructuralmente, se compone de dos barras verticales que representan las líneas de alimentación eléctrica, entre las cuales se disponen líneas horizontales llamadas "peldaños" (*rungs*). En cada peldaño se insertan contactos (que representan condiciones físicas o lógicas de entrada) conectados en serie o paralelo hacia bobinas (que representan las salidas o actuadores). Su principal ventaja radica en que es altamente intuitivo para electricistas y técnicos de mantenimiento.

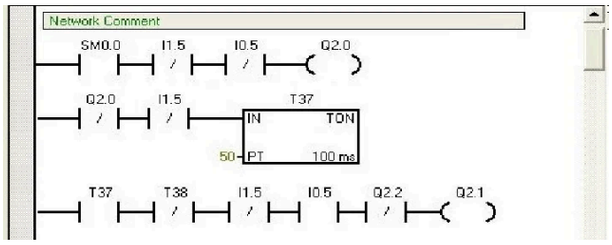


Figure 1. Ejemplo de programa en Ladder

- **SFC / GRAFCET (Gráfico Funcional de Secuencias):** Es un lenguaje de alto nivel diseñado específicamente para estructurar de forma visual el control de procesos secuenciales complejos y concurrentes. Su metodología se fundamenta en una red de **etapas**, **transiciones** y **acciones asociadas**. Las *etapas* representan un estado estable del sistema donde se ejecutan acciones específicas (por ejemplo, "bajar prensa") ; las *transiciones* son condiciones lógicas que deben cumplirse para avanzar a la siguiente etapa (por ejemplo, sensor "Prensa_Abajo" activado). Es la herramienta ideal para la descripción funcional previa a la codificación final.

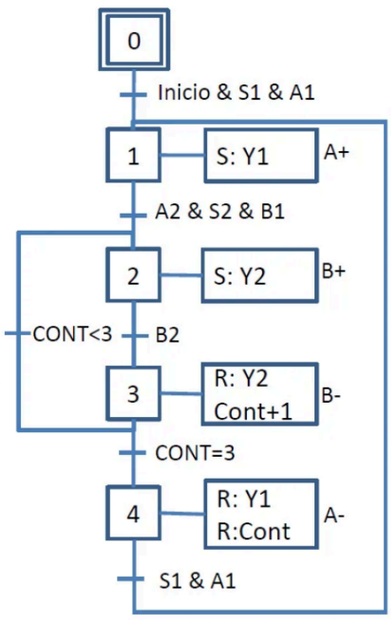


Figure 2. Ejemplo de programa en GRAFCET

- **FBD (Diagrama de Bloques de Funciones):** Este lenguaje modela el flujo de señales basándose en el principio de los diagramas de circuitos electrónicos y compuertas lógicas. El programa se construye mediante la interconexión de bloques funcionales (que pueden ser funciones lógicas básicas como AND/OR, temporizadores, contadores o bloques PID complejos). Las variables de entrada se inyectan por el lado izquierdo del bloque y los resultados se extraen por el lado derecho. Es un lenguaje sumamente eficiente para sistemas que involucran el procesamiento de señales analógicas y bucles de control continuos.

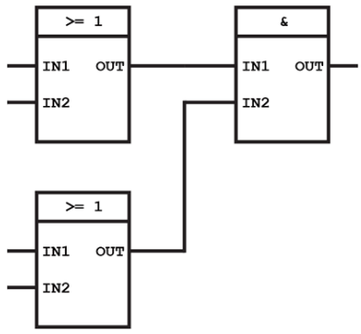


Figure 3. Ejemplo de programa en Bloques funcionales

Lenguajes de programación Textuales

- **ST (Texto Estructurado):** Es un lenguaje de programación de alto nivel cuya sintaxis y estructura textual son muy similares a lenguajes tradicionales como *Pascal* o *C*. Permite una programación sumamente matemática y estructurada mediante el uso de condicionales avanzados (`IF-THEN-ELSE`, `CASE`), bucles de repetición (`FOR`, `WHILE`, `REPEAT`) y funciones complejas. Es el lenguaje predilecto por los ingenieros cuando se deben implementar algoritmos de control numérico, procesamiento de datos masivos, manejo de strings o cálculos de ingeniería que serían extremadamente engorrosos de representar de forma gráfica.
- **IL (Lista de Instrucciones):** Es un lenguaje textual de bajo nivel, conceptualmente análogo al lenguaje *Assembler* utilizado en microcontroladores. Su ejecución es estrictamente secuencial y se basa en códigos mnemónicos simples (como `LD` para cargar, `AND` para conjunción lógica, `ST` para almacenar) que operan directamente sobre un registro acumulador central. Aunque consume muy pocos recursos de memoria y es extremadamente rápido en hardware antiguo, actualmente se encuentra en desuso dentro de la propia norma debido a que el código resultante es difícil de documentar, depurar y mantener frente a sistemas de gran escala.

Metodos de Diseno de Control Secuencial

El diseño de sistemas de control para procesos industriales automatizados, especialmente en plantas de manufactura flexible, requiere de metodologías robustas que permitan modelar el comportamiento dinámico y secuencial del sistema. Un control secuencial es aquel en el que las acciones se ejecutan paso a paso, donde la activación de una etapa posterior depende estrictamente del cumplimiento de condiciones lógicas previas (transiciones). Para garantizar un desarrollo ordenado, libre de fallas lógicas como bloqueos (*deadlocks*) o carreras críticas, la ingeniería moderna recurre a metodologías estructuradas, siendo las más importantes el GRAFCET como herramienta de diseño, y el diagrama LADDER como lenguaje de implementación final.

- **GRAFCET (SFC)**: El GRAFCET (Graphe de Commande Étape-Transition), normalizado bajo la IEC 60848 y adoptado en la IEC 61131-3 como *Sequential Function Chart* (SFC), es una metodología gráfica de modelado funcional de alto nivel. Su principal ventaja es que permite describir de forma analítica y visual el comportamiento de sistemas secuenciales concurrentes, independientemente del hardware del PLC que se vaya a utilizar.
  Un diagrama GRAFCET se fundamenta matemáticamente en las Redes de Petri y se compone estructuralmente de tres elementos esenciales:
  - **Etapas (Steps):** Representan un estado intermedio y estable del proceso automatizado. Mientras una etapa está activa, se ejecutan las acciones asociadas a ella (por ejemplo, activar el motor de la cinta o energizar una electroválvula). Se representan gráficamente mediante cuadrados numerados, distinguiendo la etapa inicial (reposo o Etapa 0) con un doble cuadrado.
  - **Transiciones (Transitions):** Son las barreras lógicas situadas entre dos etapas sucesivas. Cada transición tiene asociada una condición matemática o booleana denominada **Receptividad**, la cual se alimenta de las señales de los sensores físicos (sensores inductivos, ópticos, capacitivos o finales de carrera). Una transición se representa mediante una línea horizontal perpendicular a las líneas de unión.

- **Direccionamiento y Enlaces:** Líneas orientadas que conectan las etapas con las transiciones. La regla fundamental del GRAFCET dicta que el sistema progresa cuando una etapa previa está activa y la receptividad de la transición inmediatamente posterior se vuelve verdadera, provocando la desactivación de la etapa anterior y la activación automática de la siguiente.

Esta metodología es ideal para automatizaciones complejas ya que permite modelar estructuras avanzadas como secuencias lineales, selección de secuencia o divergencia en OR (toma de decisiones basada en condiciones, como la clasificación por tipo de material) y secuencias simultáneas o divergencia en AND.

- **LADDER (Diagrama de Contactos)**: Mientras que el GRAFCET se utiliza para la fase de análisis y diseño conceptual, el diagrama LADDER es el lenguaje gráfico de bajo nivel elegido para la traducción e implementación final de dicha lógica dentro de la memoria del controlador (PLC).
  LADDER (también conocido como lógica de contactos o esquemas de escalera) emula la topología de los antiguos tableros de control basados en relés electromecánicos e interruptores cableados. Sus características fundamentales incluyen:

  - **Estructura en Peldaños (Rungs):** El programa se lee de arriba hacia abajo y de izquierda a derecha. Consiste en dos rieles verticales de potencia imaginaria; la corriente fluye desde el riel izquierdo hacia el derecho a través de los peldaños si las condiciones de los contactos lo permiten.
  - **Contactos (Condiciones de Entrada):** Representan variables booleanas de entrada (sensores, pulsadores, bits de memoria). Se dividen en contactos *Normalmente Abiertos (NO)*, que permiten el paso de la señal si la variable es 1, y contactos *Normalmente Cerrados (NC)*, que permiten el paso si la variable es 0.
  - **Bobinas (Resultados de Salida):** Ubicadas estrictamente a la derecha de cada peldaño, representan los actuadores (motores, bobinas de electroválvulas) o bits internos de estado que se energizan una vez que se completa la continuidad lógica del peldaño.

Relación entre GRAFCET y LADDER en el desarrollo industrial: En la práctica de la ingeniería de control, el diseño nunca se realiza directamente en LADDER si el proceso es complejo. Primero se elabora el GRAFCET para entender la secuencia lógica de la planta. Posteriormente, aplicando métodos sistemáticos de conversión (como el método de asignación de memorias/marcas, el método de flancos o el uso de funciones SET/RESET), las etapas y transiciones del GRAFCET se traducen de forma directa y estructurada a peldaños LADDER. Esto asegura que el código final en el PLC sea ordenado, fácil de depurar por el personal de mantenimiento y altamente confiable ante paradas de emergencia.

Conceptos de Supervisión de Procesos

En el ámbito de la automatización industrial, la supervisión de procesos abarca el conjunto de tecnologías, protocolos y sistemas de software que permiten a los operadores e ingenieros monitorear, adquirir datos y controlar el estado de una planta productiva en tiempo real. La supervisión actúa como un puente entre el nivel de control de campo (donde operan los sensores, actuadores y PLCs) y el nivel de gestión operativa. Para este laboratorio, la comprensión de estos conceptos es fundamental, ya que la arquitectura propuesta replica de forma virtual una infraestructura de supervisión industrial completa a través de interfaces HMI y protocolos de red.

- **HMI (Human-Machine Interface)**: Una HMI es una interfaz gráfica basada en software o hardware localizado que permite la interacción directa y local entre un operador humano y una máquina o proceso específico.
  - **Función principal:** Traduce las variables complejas y los mapas de memoria del PLC (como los bits de entradas, salidas y registros analógicos) a representaciones visuales comprensibles, tales como gráficos de barras, luces piloto virtuales, animaciones de movimiento y botones táctiles.

- **SCADA (Supervisory Control and Data Acquisition)**: A diferencia de una HMI, que generalmente se limita a la supervisión local de una sola máquina o estación, un sistema SCADA es una solución de software de alto nivel diseñada para la supervisión y adquisición de datos de procesos industriales complejos a gran escala, distribuidos geográficamente o compuestos por múltiples subestaciones.

    - **Estructura Funcional:** Un SCADA integra la comunicación con múltiples controladores (PLCs) y añade funciones avanzadas como el almacenamiento de datos históricos (historizadores), la gestión centralizada de alarmas y eventos, generación de reportes de producción y herramientas de análisis de tendencias.
    - **Importancia de su Evolución:** Las plataformas modernas de SCADA (como WinCC Unified o arquitecturas basadas en Node-RED) se conectan a bases de datos relacionales y servidores en la nube para permitir el análisis de eficiencia energética y la optimización de la producción, sirviendo como base para propuestas de mejora en sistemas que inicialmente solo cuentan con una visualización básica.

- **OPC / Modbus TCP**: La supervisión y el control dependen críticamente de la robustez de los canales de comunicación industriales, los cuales aseguran que los datos entre los sensores, los actuadores, el PLC y la HMI se intercambien sin pérdidas de información y con latencias mínimas.
    - **Modbus TCP:** Es uno de los protocolos de comunicación industrial más longevos, abiertos y utilizados a nivel global debido a su simplicidad y eficiencia. Basado en el modelo Cliente/Servidor (antiguamente Maestro/Esclavo), encapsula las tramas de datos tradicionales de Modbus dentro de paquetes de red estándar TCP/IP, operando comúnmente sobre redes Ethernet industriales a través del puerto de comunicación regulado.
    - **Mapa de Memoria (Direccionamiento):** Modbus organiza la información de la planta en cuatro tablas lógicas primarias accesibles mediante direcciones numéricas:
        - **Bobinas (Coils):** Variables de lectura y escritura binaria que representan actuadores o salidas físicas del PLC (por ejemplo, electroválvulas de los cilindros neumáticos o el motor de la cinta).
        - **Entradas discretas(Discrete Inputs):** Variables de sólo lectura binaria que reflejan el estado de los sensores de campo (por ejemplo, finales de carrera o sensores de detección de material).
        - **Registros de entrada(Input Registers):** Variables de sólo lectura de 16 bits para señales analógicas.
        - **Registros de Retención(Holding Registers):** variables de lectura y escritura de 16 bits para configuración y consignas.

Justificación De Las Tecnologías Utilizadas

Python (Simulación y HMI)

Python se seleccionó como el núcleo para simular la planta y servir la HMI debido a:

- **Desarrollo Rápido y Flexibilidad**: Permite escribir el motor de físicas del simulador y las variables de estado en pocas líneas de código, facilitando la integración de hilos concurrentes para la simulación física y las comunicaciones de red.
- **Flask y Servidor Web**: Flask actúa como un servidor ligero capaz de procesar peticiones JSON y actualizar la HMI web en tiempo real (100ms) sin sobrecargar la CPU de la computadora de desarrollo.
- **Desacoplamiento**: Python ejecuta la simulación física y la comunicación de red de forma independiente, emulando la planta sin necesidad de relés, válvulas o motores reales.

OpenPLC (Control Estandarizado)

Open PLC Runtime es el entorno de ejecución industrial compatible con la norma internacional **IEC 61131-3** que se eligió para el controlador:

- **Fidelidad Industrial**: Ejecuta el código de la misma manera que lo haría un PLC comercial (como Siemens S7-1200 o S7-200), permitiendo programar en lenguajes estandarizados como Structured Text.
- **Portabilidad y determinismo**: Corre como un servicio local en Linux y proporciona herramientas de compilación en C + + que respetan los ciclos de scan deterministas.

Modbus TCP (Protocolo de Comunicación)

Modbus TCP se empleo como el bus de comunicación industrial virtual:

- **Simplicidad**: Mapea variables a través de registros sencillos (Coils para salidas y Discrete Inputs para entradas) muy fáciles de programar.
- **Compatibilidad**: Es soportado de forma nativa por Open PLC y fácilmente implementado en Python a través de la librería `pyModbusTCP`.
- **Arquitectura Cliente-Servidor**: Permite modelar a la planta como un Servidor/Esclavo (Python) que espera de forma pasiva las órdenes del Controlador Cliente/Maestro (OpenPLC), reflejando la estructura real de E/S distribuidas.

Configuración Del Entorno De Simulación

Arquitectura del Sistema Propuesto

El flujo de información se organiza en una arquitectura distribuida localmente:

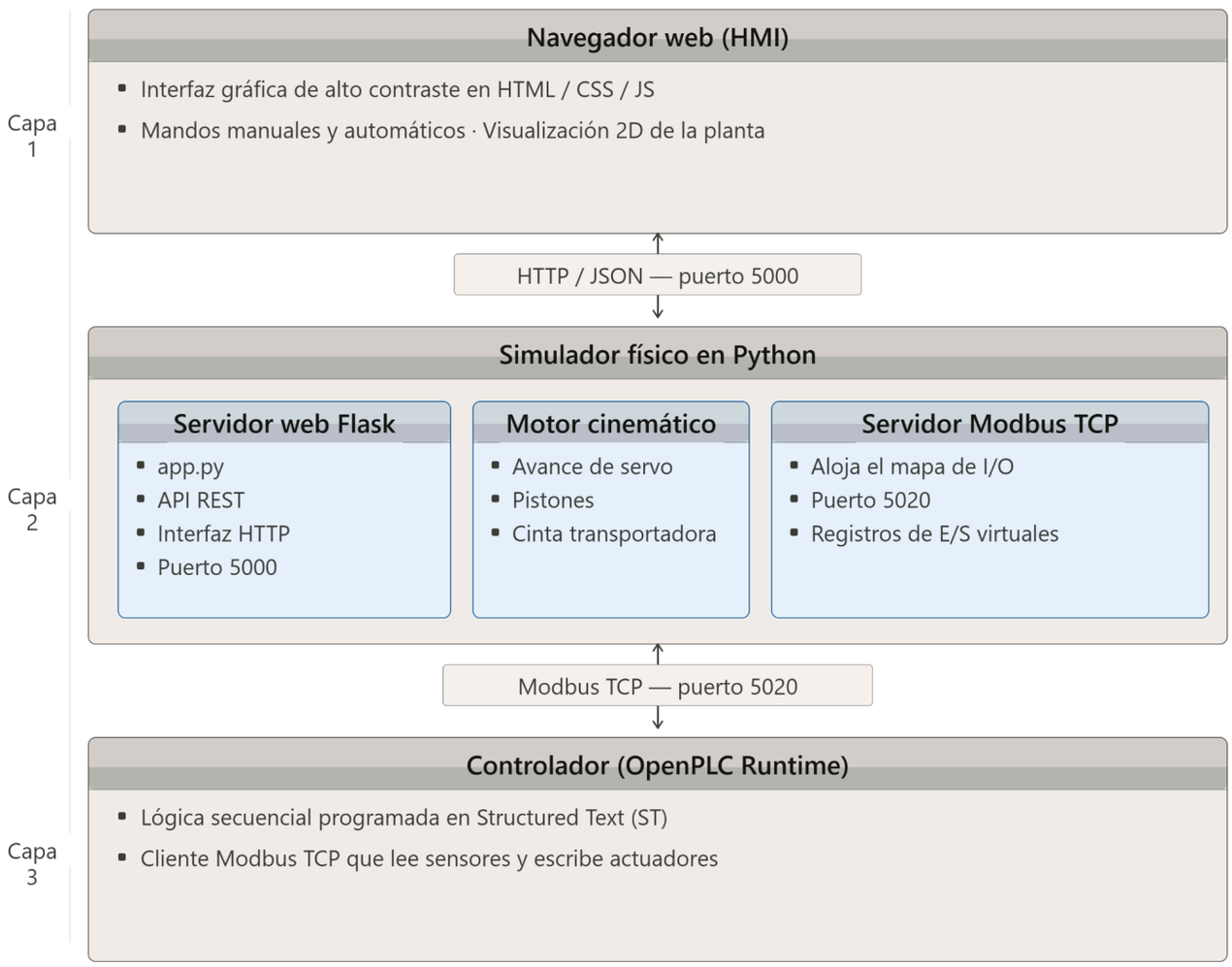


Figure 4. Arquitectura del sistema en 3 capas

Configuración del Dispositivo Remoto en Open PLC Editor

Para conectar OpenPLC con el simulador, se añadió un **Remote Device** en la configuración del editor de OpenPLC. Dado que la planta se modela como un esclavo Modbus TCP, se parametrizan las siguientes propiedades:

- **Device Type**: Generic Modbus TCP Device
- **IP Address**: `127.0.0.1` (Localhost para ejecucion en la misma maquina)
- **Port**: `5020` (Puerto configurado en el servidor pyModbusTCP del simulador)
- **Slave ID**: `1`
- **Update Rate**: `50 ms` (Intervalo de actualización periodico para garantizar respuesta rapida)

Dentro de este dispositivo se configuraron dos tablas de mapeo de variables:

1. **Discrete Inputs (Read)**: Offset de lectura desde `0`, longitud de `40` registros (mapeados localmente a `%IX0.0` en adelante).
2. **Coils (Write)**: Offset de escritura desde `0`, longitud de `40` registros (mapeados localmente a `%QX0.0` en adelante).

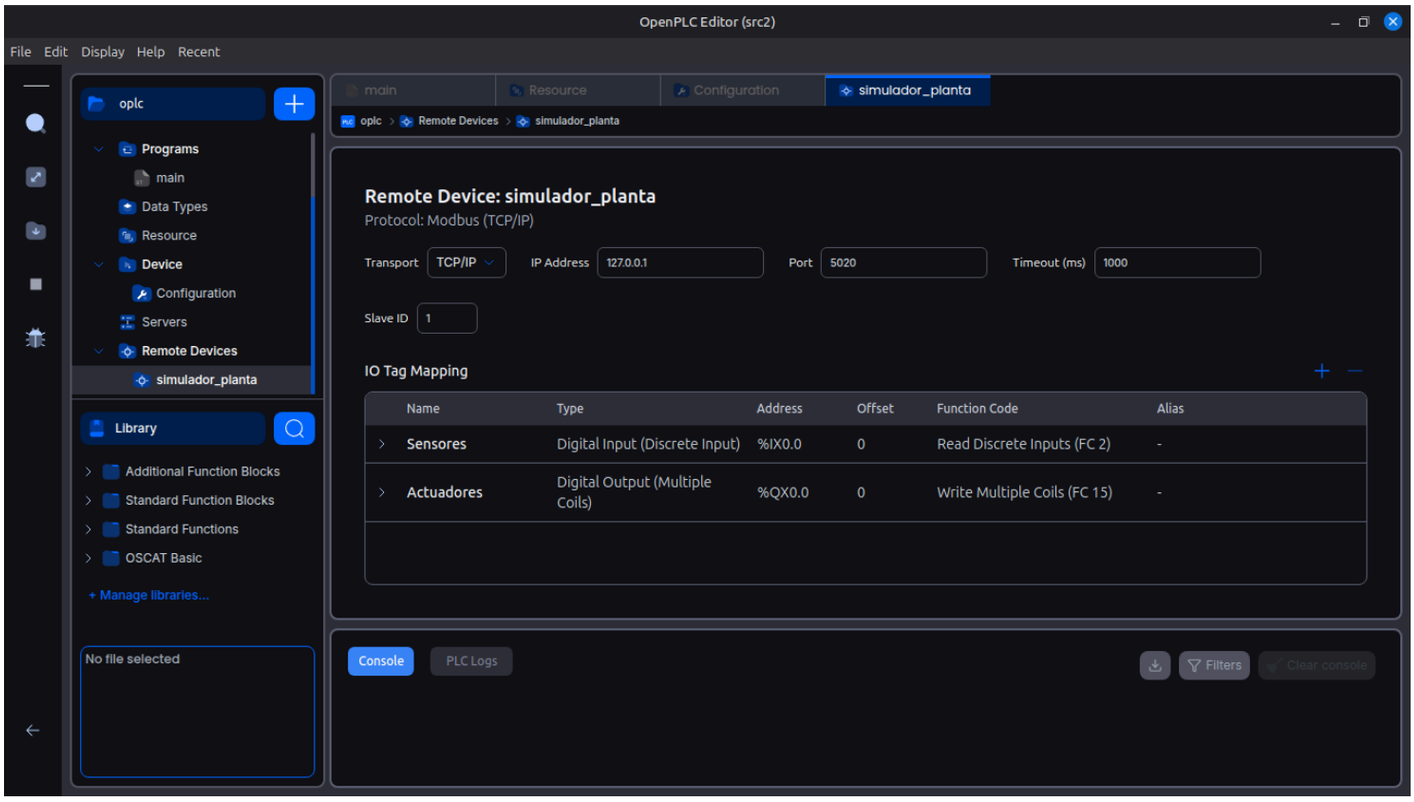


Figure 5. Ventana del editor de OpenPLC que muestra los parámetros del Remote Device de tipo Modbus TCP configurado en IP 127.0.0.1 y puerto 5020, visualizando las tablas de mapeo de 40 puntos para coils e inputs

Tabla de Direccionamiento Modbus (Mapa de Memoria)

Bobinas (Coils - Actuadores / Salidas PLC)

Las salidas Modbus escritas por el PLC controlan directamente el estado físico de los actuadores del simulador.

Dirección Modbus	Símbolo	Dirección Local PLC	Estación	Funcion
0	V_Cil_Peq	%QX0.0	U1	Sujetador de Columna (Pistón retenedor)
1	V_Cil_Lrg	%QX0.1	U1	Empujador de Pieza U1

Dirección Modbus	Símbolo	Dirección Local PLC	Estación	Funcion
8	V_Prensa	%QX1.0	U2	Cilindro de Prensa de Procesamiento
16	V_Cil_Ensa	%QX2.0	U3	Empujador de Ensamble de Piezas
24	V_Cil_Vert	%QX3.0	U4	Bajar Brazo Vertical U4 (1=Bajar, 0=Subir)
25	V_Pinza	%QX3.1	U4	Cerrar Pinza del Manipulador U4
26	Servo_Go_U1	%QX3.2	U4	Ordenar al Servo viajar a posición de U1
27	Servo_Go_U2	%QX3.3	U4	Ordenar al Servo viajar a posición de U2
28	Servo_Go_U3	%QX3.4	U4	Ordenar al Servo viajar a posición de U3
29	Servo_Go_U5	%QX3.5	U4	Ordenar al Servo viajar a posición de U5
32	Motor_Cinta	%QX4.0	U5	Encender Motor de la Cinta Transportadora
33	V_Deflector_R1	%QX4.1	U5	Extender Deflector neumático Rampa 1
34	V_Deflector_R2	%QX4.2	U5	Extender Deflector neumático Rampa 2
35	V_Deflector_R3	%QX4.3	U5	Extender Deflector neumático Rampa 3

Table 1. Tabla de Mapeo de salidas del PLC

Entradas (Discrete Inputs - Sensores / Entradas PLC)

Las entradas Modbus leídas por el PLC informan sobre la posición mecánica y detección de sensores en la planta.

Dirección Modbus	Símbolo	Dirección Local PLC	Estación	Funcion
0	Cil_Peq_Ext	%IX0.0	U1	Piston retenedor U1 extendido

Dirección Modbus	Símbolo	Dirección Local PLC	Estación	Funcion
1	Cil_Peq_Ret	%IX0.1	U1	Piston retenedor U1 retraído
2	Cil_Lrg_Ext	%IX0.2	U1	Pistón empujador U1 extendido
3	Cil_Lrg_Ret	%IX0.3	U1	Pistón empujador U1 retraído
4	Presencia_Inf	%IX0.4	U1	Pieza dosificada lista en base U1
8	Prensa_Arriba	%IX1.0	U2	Cilindro de Prensa en posición superior
9	Prensa_Abajo	%IX1.1	U2	Cilindro de Prensa en posición inferior
16	Cil_Ensa_Ret	%IX2.0	U3	Cilindro de ensamble U3 retraído
17	Cil_Ensa_Ext	%IX2.1	U3	Cilindro de ensamble U3 extendido
24	Cil_Vert_Arriba	%IX3.0	U4	Brazo mecánico U4 en posición superior
25	Cil_Vert_Abajo	%IX3.1	U4	Brazo mecánico U4 en posición inferior
26	Pinza_Cerrada	%IX3.2	U4	Pinza cerrada detectando presencia de pieza
27	Servo_En_U1	%IX3.3	U4	Servo posicionado en Estacion Alimentación
28	Servo_En_U2	%IX3.4	U4	Servo posicionado en Estación Procesamiento
29	Servo_En_U3	%IX3.5	U4	Servo posicionado en Estación Ensamblaje
30	Servo_En_U5	%IX3.6	Servo	Servo posicionado en Estación Clasificación
32	Sens_Objeto_Ent	%IX4.0	U5	Pieza detectada al ingreso de la cinta U5

Dirección Modbus	Símbolo	Dirección Local PLC	Estación	Funcion
33	Sens_Metal	%IX4.1	U5	Sensor inductivo detecta pieza de Metal
34	Sens_Mixto	%IX4.2	U5	Sensor óptico detecta pieza Mixta
35	Sens_Plastico	%IX4.3	U5	Sensor capacitivo detecta pieza de Plástico
36	Deflector_R1_Ext	%IX4.4	U5	Deflector neumático Rampa 1 extendido
37	Deflector_R2_Ext	%IX4.5	U5	Deflector neumático Rampa 2 extendido
38	Deflector_R3_Ext	%IX4.6	U5	Deflector neumático Rampa 3 extendido

Table 2. Tabla de Mapeo de entradas del PLC

Desarrollo Del Laboratorio Y Secuencia De Control

Proceso Secuencial de la Planta

El funcionamiento automático de la planta XK335B se diseñó como una secuencia lineal coordinada dividida en las siguientes etapas lógicas:

1. **Dosificación en Alimentación (U1)**: Cuando se inicia el proceso, el pistón superior (Cil Peq) sujeta la columna de stock. Luego, el pistón empujador inferior (Cil_Lrg) se extiende para empujar la pieza base hacia la rampa de entrega y después retorna. Finalmente, el pistón superior retorna para dejar caer la siguiente pieza en stock.
2. **Viaje y Agarre (U4)**: El carro del servo de transporte viaja de forma horizontal a la posición de la Estación U1 (20000 pulsos). Al llegar, el brazo vertical baja, se activa la pinza para sujetar la pieza dosificada, y el brazo vertical vuelve a subir.
3. **Transporte a Prensa (U4 -> U2)**: El servo traslada la pieza a la Estación de Procesamiento U2 (44000 pulsos). Al posicionarse, el brazo vertical baja, se desenergiza la pinza para liberar la pieza en la base de la prensa, y el brazo vertical sube para despejar la zona.
4. **Procesado por Prensa (U2)**: Con el brazo de transporte fuera de peligro, el cilindro neumático de la prensa baja, sella la pieza por presión durante un intervalo de tiempo y retorna a su posición superior.
5. **Transporte a Ensamble (U4 -> U3)**: El brazo de transporte recoge la pieza ya prensada en U2 y la traslada a la Estación de Ensamblaje U3 (81500 pulsos). Libera la pieza base sobre la mesa de ensamble y sube.
6. **Ensamblado (U3)**: Un pistón de ensamble lateral empuja una tapa (pieza pequeña) sobre la pieza base colocada en la mesa de ensamble, realizando la unión mecánica por presión, y retorna a su posición inicial.
7. **Transporte y Clasificación (U4 -> U5)**: El manipulador U4 recoge la pieza ensamblada de U3 y la deposita en el extremo inicial de la cinta transportadora de U5 (107000 pulsos).
8. **Selección y Desvío (U5)**: La cinta transportadora se enciende y la pieza avanza. Según el tipo de material detectado por los sensores de clasificación de U5 (inductivo, óptico o capacitivo), el deflector

neumático correspondiente (Rampa 1, 2 o 3) se extiende para desviar la pieza a su respectiva rampa de salida.

Implementación de la Lógica en Structured Text (ST)

La programación del controlador en OpenPLC se desarrolló en Structured Text (ST). Se implementó una máquina de estados de control mediante la instrucción condicional `CASE Step OF ... END_CASE`.

Durante las fases iniciales de pruebas con la planta, se identificaron y resolvieron tres problemas críticos de programación para garantizar que el comportamiento del gemelo digital sea robusto:

1. **Problema de Retención de Salidas (Evitado por Reset de Scan)**: En los lenguajes de PLC, las salidas permanecen activas si la lógica secuencial no las apaga explícitamente. Para evitar que actuadores de etapas previas queden energizados (por ejemplo, mantener el cilindro prensa o el servo activado indefinidamente), se programó un bloque de inicialización al inicio de cada scan del PLC que forza a `FALSE` todas las bobinas físicas. Si una etapa requiere que una salida esté activa, la máquina de estados la vuelve a colocar en `TRUE` durante ese mismo ciclo de ejecución, garantizando un funcionamiento seguro y determinista.
2. **Evaluación Incondicional de Temporizadores (TON)**: Se utilizó el bloque temporizador de tipo `TON` para controlar los tiempos de retardo (ej. tiempo de cierre de la pinza o duración de la clasificación). Inicialmente, la llamada al temporizador se realizaba dentro del bloque condicional `CASE`. Esto provocó que el temporizador "congelara" su valor y no contará el tiempo correctamente al salir de la etapa. Para resolverlo, la declaración de llamada a los temporizadores `Timer_Pinza()` y `Timer_Seleccion()` se movió al final del programa, fuera de la estructura del `CASE`, asegurando que el PLC actualiza el reloj interno de los temporizadores de forma incondicional en cada ciclo de scan.
3. **Solución de Atascamiento en la Cinta U5**: Cuando la pieza llegaba a la unidad de clasificación, se atascaba en medio de la cinta porque el motor se apagaba inmediatamente al retraer el deflector. Se agrego una etapa de temporización de 3 segundos (`Timer_Seleccion`) que mantiene el motor de la cinta y la bobina del deflector activos tras la detección, permitiendo que la pieza se deslice completamente por la rampa de gravedad antes de apagar la cinta y retraer el pistón deflector.

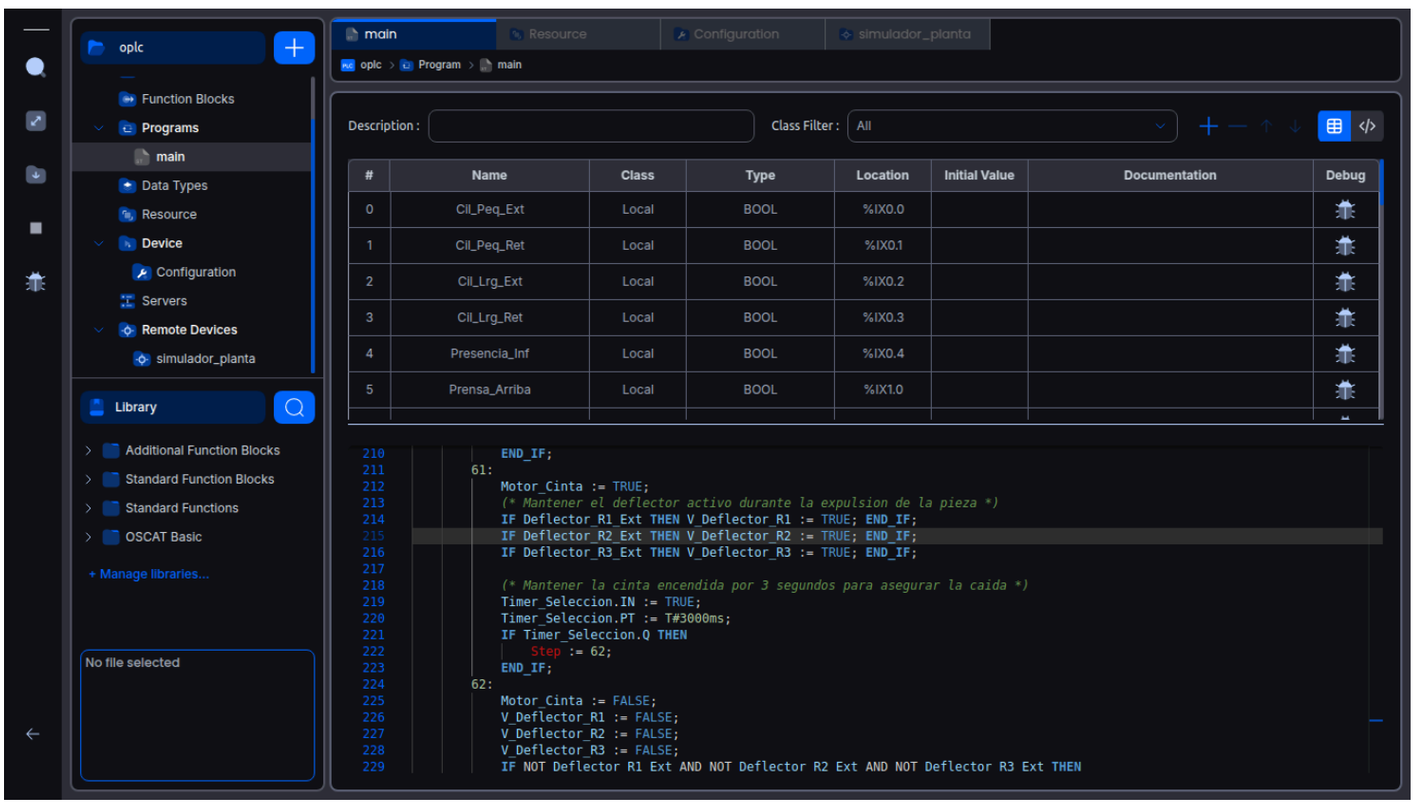


Figure 6. Captura de pantalla de la sección de edición de código Structured Text en el editor de OpenPLC, visualizando las variables globales y el bloque principal de la máquina de estados implementada con CASE

Pruebas en Modo Manual

Para comprobar la instalación mecánica y el mapa de E/S virtuales, se desactivo la casilla "Control Automático" en el panel HMI. Esto habilitó los botones de mandos manuales del HMI, lo que permitió probar de forma individual cada pistón neumático, abrir/cerrar la pinza y mover el carro del servo. La prueba manual válida muestra que el servidor Modbus TCP responde de forma inmediata a los cambios escritos desde la interfaz web sin interferir con la lógica del PLC (el cual permanece en la etapa de reposo Step 0).

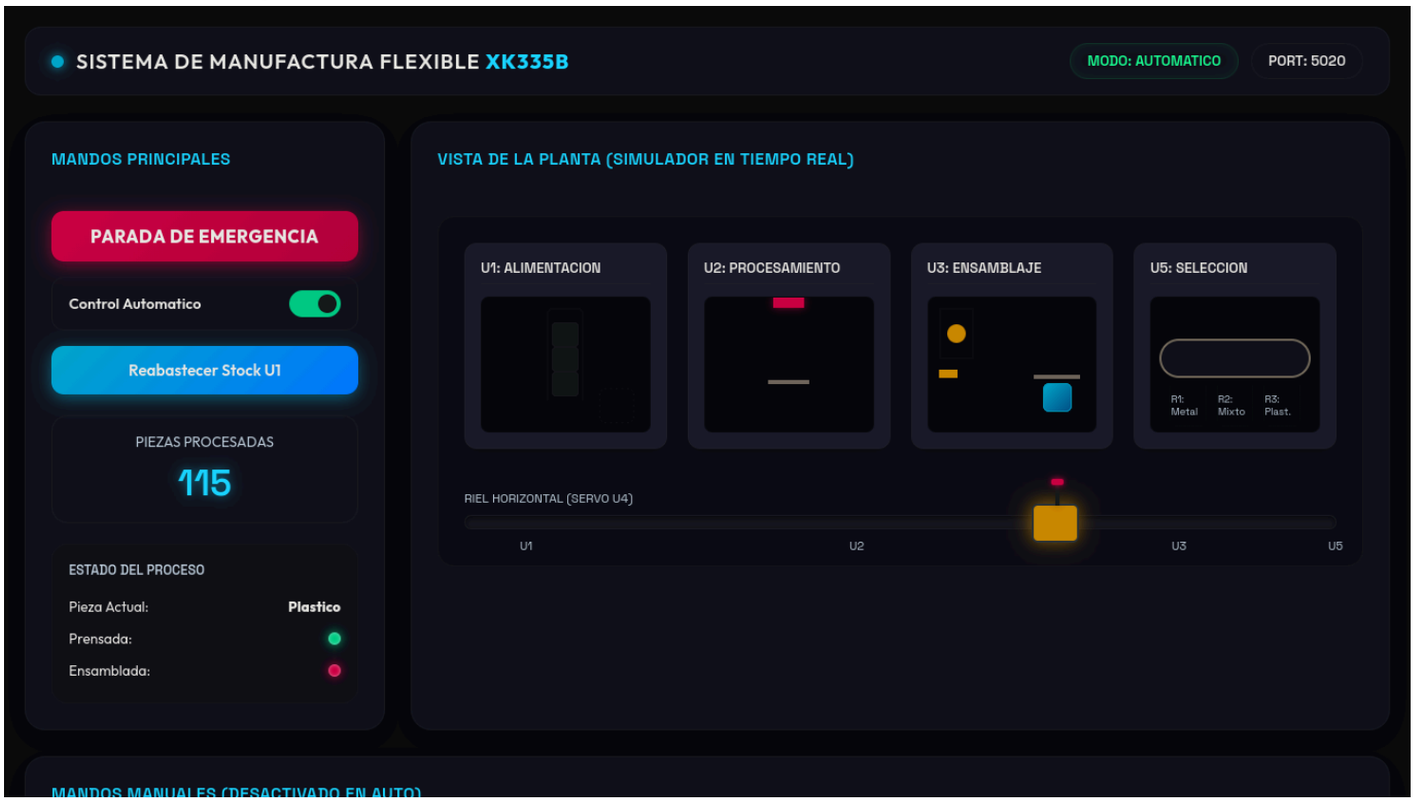


Figure 7. Captura de pantalla: Modo Manual

Pruebas de Parada de Emergencia

Se comprobó la seguridad del sistema mediante la pulsación del botón rojo de Parada de Emergencia en el HMI. Al activarse la parada de emergencia:

- El simulador en Python interrumpe la actualización normal y escribe `FALSE` en todas las bobinas del banco de datos Modbus.
- En consecuencia, todos los pistones del simulador retornan a su posición de reposo de forma automática y el motor de la cinta se detiene de inmediato.
- El PLC detecta la caída de señales del entorno y reinicie su máquina de estados interna a la etapa `Step 0`, quedando bloqueado hasta que se restablezca la seta de emergencia.

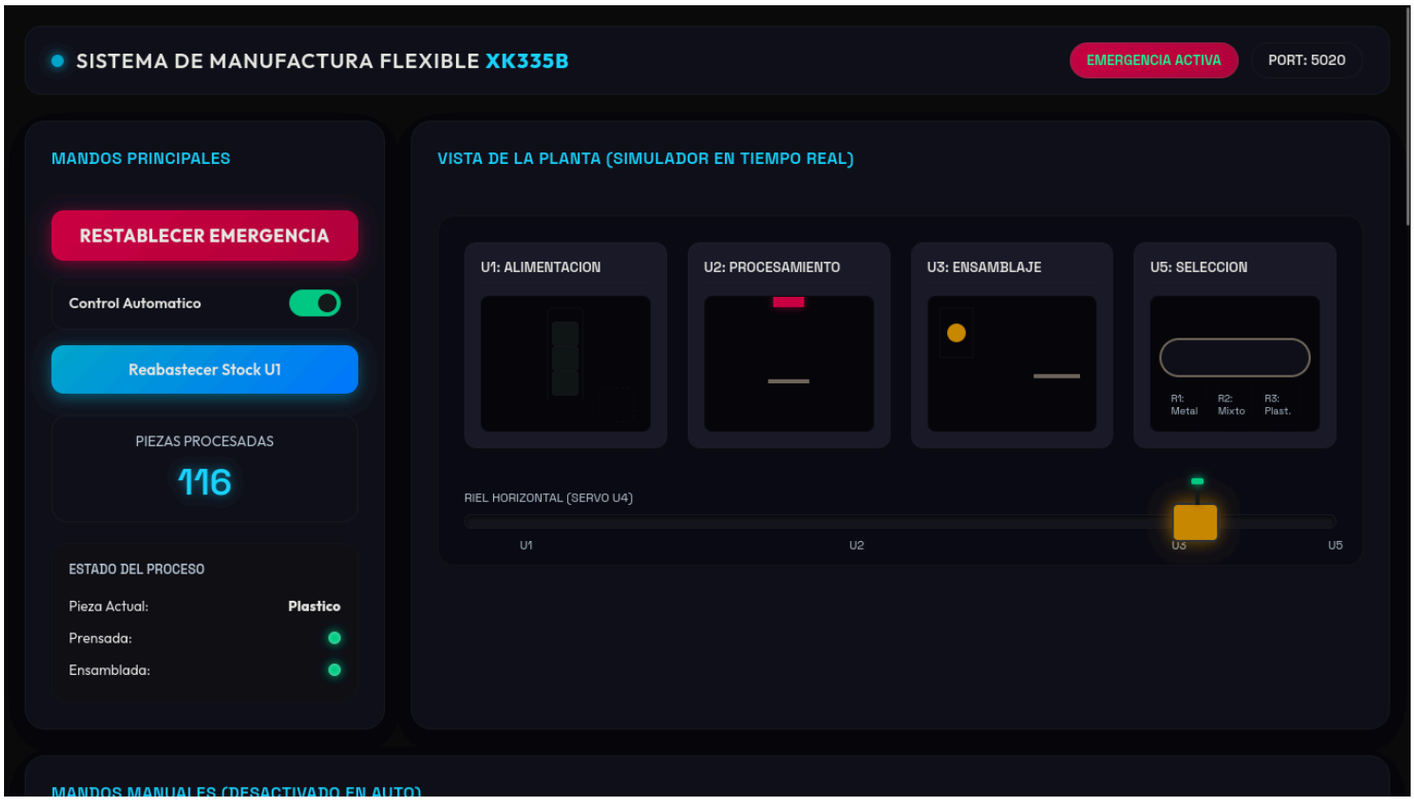


Figure 8. Captura de pantalla: Parada de emergencia

Validacion Y Analisis De Resultados

Tiempos de Respuesta Cinemáticos

Para lograr una simulación representativa de la planta real, el motor cinemático del simulador en Python calcular las posiciones mecánicas en intervalos periódicos, emulando las siguientes velocidades físicas:

- **Cilindros Pequenos (Dosificador U1, Ensamble U3, Deflectores U5)**: Tiempo de carrera completa configurado en `0.5 segundos` (100% de extensión).
- **Cilindro Empujador Largo (Dosificador U1)**: Tiempo de carrera completa configurado en `0.8 segundos`.
- **Cilindro Prensa (Estación U2)**: Tiempo de descenso/ascenso configurado en `1.0 segundo`.
- **Eje Servo de Transporte (U4)**: El servo se simuló con una velocidad lineal de `30000 pulsos por segundo` (la posición máxima de U5 a 107000 pulsos toma aproximadamente 3.5 segundos desde el origen).

El PLC virtual demostró la capacidad de reaccionar de forma sincronizada con estas velocidades, transicionando de etapa en el instante preciso en que los sensores correspondientes (`Cil_Lrg_Ext`, `Servo_En_U3`, etc.) pasan a estado lógico `TRUE` en la memoria Modbus compartida.

Estabilidad de la Comunicación Modbus TCP

La interconexión local entre el PLC y la planta a través del bucle de retorno local (Loopback en `127.0.0.1:5020`) mostro una excelente estabilidad:

- **Latencia de Red**: Los tiempos de ida y vuelta (Round Trip Time) de los paquetes Modbus TCP se mantuvieron constantes.

- **Tasa de Refresco (Update Rate)**: Configurado a `50 ms` en el Remote Device de OpenPLC, permitiendo una actualización de datos más que suficiente para no introducir retardos significativos frente a los movimientos mecánicos de la planta.
- **Pérdida de Paquetes**: Cero errores de comunicación durante ciclos continuos de ejecución de más de 1 hora de duración.

Limitaciones Y Comparación Con La Planta Real

A pesar del éxito en el comportamiento de la planta simulada, existen diferencias importantes respecto al comportamiento de la planta física real XK335B:

Característica / Parámetro	Planta Real XK335B	Simulación Virtual (Etapa 2)
Eje Servo U4	Posee rampas de aceleración y desaceleración controladas por la parametrización interna del servo drive.	Se mueve con una velocidad lineal constante (sin rampas de aceleración) en el simulador Python.
Giro de Garra U4	Realiza giros físicos de 90 y 180 grados mediante una electroválvula giratoria neumática.	Simplificado en el HMI 2D, representando de forma fija orientada hacia arriba sin mostrar giro mecánico.
Control de Velocidad U5	El motor de la cinta es controlado mediante un variador de frecuencia análogo (AQW0 a 0-10V) y control de bucle cerrado.	Se simuló con velocidad de cinta constante mediante una bobina digital binaria (Motor_Cinta).
Física de Sensores	Los sensores inductivos y ópticos reales sufren de ruido eléctrico y falsos disparos por suciedad o luz.	Los sensores lógicos virtuales son ideales y cambian a TRUE instantáneamente según la posición geométrica de la pieza.

Table 3. Comparación entre la planta real y la simulada.

Propuestas De Mejora Y Trabajo Futuro

Para aproximar aún más el gemelo digital a la planta real, se proponen las siguientes mejoras técnicas para etapas posteriores:

1. **Modelado Cinemático con Aceleración**: Incorporar en el código de Python perfiles de aceleración trapezoidales o senoidales para emular las fuerzas de inercia y rampas de velocidad del servo real.
2. **Control Análogo de la Cinta Transportadora**: Implementar la lectura de la palabra de salida análoga del PLC (`AQW0`) en el simulador para controlar la velocidad del motor de la cinta de forma proporcional, integrando una aproximación al variador de frecuencia real.
3. **SCADA Distribuido**: Desarrollar una interfaz de supervisión utilizando herramientas SCADA profesionales como WinCC Unified, Node-RED Dashboard o Ignition, permitiendo registrar gráficos de tendencias históricas y bases de datos de alarmas de manera industrializada.

Conclusiones

- Se logró diseñar, implementar y validar de forma exitosa el gemelo digital de la planta flexible XK335B en su Etapa 2, logrando un control determinista desde OpenPLC Runtime sobre un entorno simulado de forma estable y fluida vía Modbus TCP.
- La selección de Structured Text (ST) como lenguaje de control secuencial facilitó la lectura, depuración e implementación de la máquina de estados de 7 etapas en comparación con un diagrama LADDER convencional, permitiendo resolver de forma estructurada los temporizadores y el reset incondicional de salidas.
- El protocolo Modbus TCP confirmó ser una excelente elección por su robustez, permitiendo aislar el controlador logico de la simulación física y garantizando que el programa de control desarrollado en Structured Text sea directamente exportable a un PLC físico compatible con IEC 61131-3 sin requerir modificaciones en la lógica.